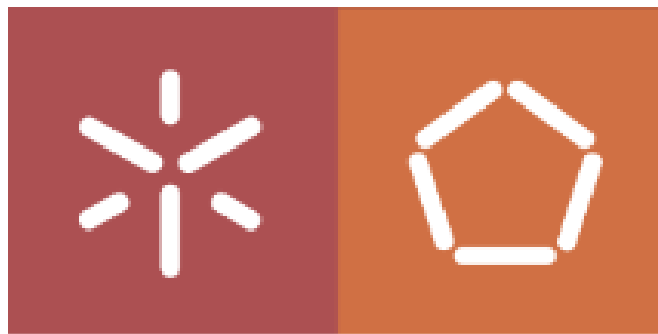


Inteligência Artificial

Trabalho Prático: Otimização de gestão de frotas

Janeiro 2026



Universidade do Minho

Escola de Engenharia

Grupo: 8

Realizado por: Juliana Sofia Vaz da Silva, a105572

Sofia Beatriz Miranda Couto, a106925

Soraia Filipa Ribeiro Pereira, a106806

Tiago Ferreira Soares, a97381

Avaliação por pares:

A105572 Juliana Sofia Vaz da Silva	DELTA = 0
A106925 Sofia Beatriz Miranda Couto	DELTA = 0
A106806 Soraia Filipa Ribeiro Pereira	DELTA = 0
A97381 Tiago Ferreira Soares	DELTA = 0

Índice

Avaliação por pares:	1
Índice	1
1. Descrição do Problema	2
2. Formulação do Problema	3
2.1. Tipo de Problema.....	3
2.2. Representação do Estado.....	3
2.3. Estado Inicial.....	3
2.4. Operadores de mudança de estado.....	4
2.4.1. Atribuir veículo a pedido.....	4
2.4.2. Recarregar/ Reabastecer veículo:.....	5
2.4.3. Rejeitar Pedido (Timeout):.....	5
2.4.4. Concluir Atendimento de Pedido.....	5
2.4.5. Atualizar Posição do Veículo.....	6
2.4.6. Gerar Novo Pedido.....	6
2.5. Teste Objetivo.....	6
2.6. Custo da Solução.....	6
3. Implementação	7
3.1. Algoritmos de procura não informada.....	7
3.1.1. Bfs (Procura em largura).....	7
3.1.2. Dfs (Procura em profundidade).....	8
3.1.3. Iterativo.....	8
3.1.4. Custo Uniforme.....	8
3.2. Algoritmos de procura informada.....	9
(Hd) - heurística da distância de manhattan.....	9
3.2.1. A Star.....	9
3.2.2 Greedy.....	9
4. Testes	10
4.1. Comparação dos algoritmos de procura.....	10
4.2. Teste.....	10
5. Interface gráfica	11
6. Conclusão	13

1. Descrição do Problema

A TaxiGreen é uma empresa de transporte urbano que opera numa área fictícia composta por diferentes zonas como a Old Town (área bastante movimentada) e por exemplo as áreas residenciais, que são zonas calmas e pouco movimentadas. A rede consta de vários pontos de recolha (R) dispersos estrategicamente, bem como diversas estações de serviço que abastecem tanto viaturas elétricas como a combustão, o que facilita bastante a Taxigreen que tem um frota mista. Os pontos de recarga elétricos (E) têm limite de 2 veículos por vez e um tempo médio de recarga considerável. Já os pontos de abastecimento a combustão (G), contam com um limite de 4 veículos por vez e um tempo médio de reabastecimento significativamente inferior ao dos veículos elétricos.

A empresa é sediada no nó E1 que funciona como sede, depósito e estação de carregamento, sendo o ponto de partida da sua frota. Assim, quando um veículo inicia um serviço a partir da sede, o mesmo apresenta a sua autonomia máxima.

O objetivo da empresa é otimizar a gestão da sua frota garantindo que todos os pedidos de transporte sejam atendidos dentro de um período de tempo aceitável, respeitando as limitações específicas de cada tipo de veículo. Os veículos elétricos têm autonomia limitada e necessitam de recarga periódica em estações específicas, enquanto os veículos a combustão podem reabastecer mais rapidamente, mas têm custos operacionais superiores e maior impacto ambiental.

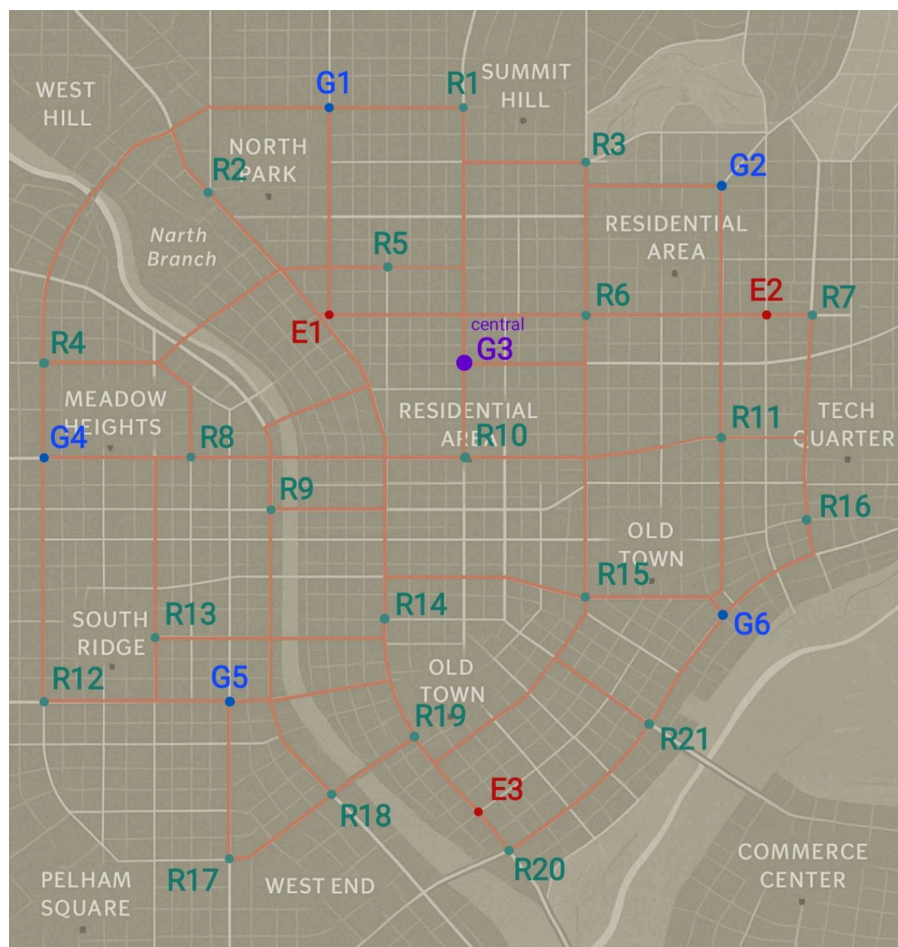


Figura 1: Grafo utilizado

2. Formulação do Problema

2.1. Tipo de Problema

O cenário contextual da TaxiGreen caracteriza-se como um problema **não-determinístico**, uma vez que o sistema evolui consoante fatores imprevisíveis e dinâmicos, tais como a chegada de novos pedidos, trânsito, variação de autonomia e disponibilidade de estações de recarga/reabastecimento.

Por outro lado, o espaço de estados é conhecido e finito, sendo definido pelo conjunto dos veículos da frota e outras variáveis de contexto descritas na modelação. Desta forma, trata-se de um **problema de contingência**.

2.2. Representação do Estado

No contexto deste problema, um estado pode ser definido como $E = (t, V, P)$, onde:

- t é o instante de tempo t
- V é o conjunto de veículos da frota, cada um formado por:
 - Id (matrícula);
 - Tipo (elétrico ou combustão);
 - Autonomia máxima (em km);
 - Autonomia atual (em km);
 - Capacidade de passageiros;
 - Tempo de recarga ou abastecimento;
 - Localização (nó do grafo onde se encontra);
 - Estado (Disponível, Ocupado ou Abastecer);
 - Custo operacional (por km);
 - Posição (coordenadas);
 - Emissões (g/km);
- P é o conjunto de pedidos, cada um formado por:
 - Id;
 - Localização de origem;
 - Localização de destino;
 - Número de passageiros;
 - Horário pretendido;
 - Tempo de espera máximo;
 - Prioridade (Normal, premium ou urgente);
 - Preferência ambiental (elétrico ou combustão);
 - Estado (Pendente, Atribuído, Em transporte, Concluído ou Rejeitado).

2.3. Estado Inicial

O estado inicial pode ser representado por:

$$E = (0, V, \varnothing)$$

Onde V corresponde à lista de veículos da frota definida consoante a informação apresentada na Tabela 1. Todos os veículos encontram-se na central, representada pelo nó El no grafo, como observável na Figura 1.

Id	Tipo de Veículo	Autonomia max (km)	Capacidade Passageiros	Tempo de abastecimento/carga (hours)	Custo por km	Emissões por km
AA-01-EE	Elétrico	350	5	1.3	0.05	0
BB-02-EE	Elétrico	300	4	2	0.06	0
CC-03-EE	Elétrico	400	5	1.7	0.05	0
DD-04-CC	Combustão	800	5	0.1	0.15	120
EE-05-CC	Combustão	700	7	0.12	0.18	160
FF-06-CC	Combustão	600	5	0.8	0.14	135
GG-07-CC	Combustão	750	4	0.1	0.15	145

Tabela 1: Informação relativa aos veículos da frota

2.4. Operadores de mudança de estado

Uma mudança de estado resulta de um dos seguintes operadores:

2.4.1. Atribuir veículo a pedido

Atribui um pedido pendente a um veículo disponível.

- Pré-requisitos:
 - Veículo $v \in V$ com estado == DISPONIVEL;
 - Pedido $p \in P$ com estado == PENDENTE;
 - Capacidade de passageiros do veículo $\geq$ Número de passageiros do pedido;
 - Tipo de veículo deve satisfazer a preferência ambiental do pedido.
- Mudanças de estado:
 - Veículo fica com estado OCUPADO;
 - Localização do veículo é atualizada ao longo da rota;
 - Autonomia atual do veículo diminui conforme a distância percorrida;
 - Pedido fica com estado ATRIBUIDO;
 - Métricas acumuladas são atualizadas: custo_operacional_acumulado, emissoes_totais, distancia_vazio_total;
 - Veículo é removido dos veículos disponíveis.

2.4.2. Recarregar/ Reabastecer veículo:

Envia um veículo para a estação de recarga/abastecimento mais próxima.

- Pré-requisitos:
 - Veículo $v \in V$ com estado == DISPONIVEL;
 - Veículo com 20% de autonomia;
 - Existe estação/posto disponível;
 - Existe uma rota válida para a estação.
- Mudanças de estado:
 - Veículo com estado ABASTECENDO;
 - Localização do veículo atualiza-se durante a deslocação;
 - Ao chegar à estação a autonomia atual do veículo passa a ser igual à autonomia máxima após o seu tempo_recarga_abastecimento;
 - No fim do carregamento, o veículo fica com estado DISPONIVEL;
 - Métricas acumuladas são atualizadas: custo_operacional_acumulado, emissoes_totais, distancia_vazio_total.

2.4.3. Rejeitar Pedido (Timeout):

Um pedido é automaticamente rejeitado quando excede o tempo máximo de espera.

- Pré-requisitos:
 - Pedido $p \in P$ com estado == PENDENTE;
 - O tempo que um pedido fica em espera é maior que o seu tempo máximo de espera.
- Mudanças de estado:
 - Pedido fica com estado REJEITADO;
 - Pedido é removido da lista de pedidos pendentes;
 - O número de pedidos rejeitados e número de pedidos completos é incrementado.

2.4.4. Concluir Atendimento de Pedido

Conclusão bem-sucedida de um pedido (ocorre automaticamente após o transporte).

- Pré-requisitos:
 - Veículo $v \in V$ com estado == OCUPADO;
 - Pedido $p \in P$ com estado == EM_TRANSPORTE;
 - Veículo chegou ao destino.
- Mudanças de estado:
 - Pedido fica com estado CONCLUIDO;
 - Veículo fica com estado DISPONIVEL;
 - Número de pedidos completos é incrementado;

2.4.5. Atualizar Posição do Veículo

Movimentação incremental de um veículo ao longo de uma rota.

- Pré-requisitos:
 - Veículo $v \in V$ com estado == OCUPADO;
 - Rota atribuída não vazia;
 - Veículo tem autonomia atual > 0 .
- Mudanças de estado:
 - Veículo atualiza para o novo nó na rota;
 - A autonomia atual do veículo é atualizada e são subtraídos os kms feitos;

2.4.6. Gerar Novo Pedido

Adição de um novo pedido ao sistema.

- Pré-requisitos:
 - Número de pedidos gerados é menor que o número máximo de pedidos;
 - Rota atribuída não vazia;
- Mudanças de estado:
 - Veículo atualiza para o novo nó na rota;
 - A autonomia atual do veículo é atualizada e são subtraídos os kms feitos;

2.5. Teste Objetivo

O teste objetivo consiste em verificar dois pontos essenciais do estado final do sistema: o estado dos pedidos e dos veículos. Ou seja, pretende-se verificar que todos os pedidos submetidos foram processados pelo sistema, independentemente de terem sido concluídos com sucesso ou rejeitados pelo elevado tempo de espera e que todos os veículos estão com estado disponível.

2.6. Custo da Solução

Devido à natureza deste problema (contingência), o custo da solução é variável, dependendo de fatores como o tempo de espera, o custo operacional, o número de pedidos não atendidos e a distância percorrida sem passageiros. Os custos operacionais variam consoante o consumo do veículo e a distância percorrida. Alterações nas rotas ou atrasos causados pelo trânsito podem levar a percursos mais longos, o que também se reflete numa maior distância percorrida sem transporte de passageiros e um maior tempo de espera. As emissões de carbono também variam em função do tipo de veículo e da distância percorrida.

Tendo em conta que cada um destes fatores contém um peso que representa a sua importância relativa no custo total, consideremos que:

- O é a soma dos custos operacionais, com peso α
- T é o tempo de espera total, com peso β
- E são as emissões de carbono estimadas, com peso γ

- D é a distância total percorrida sem transporte de passageiros, com peso $\square$
- P_R é o número de pedidos rejeitados, com peso λ

Assim, o custo total da solução, C , pode ser representado por:

$$C = \alpha \cdot O + \beta \cdot T + \gamma \cdot E + \delta \cdot D + \lambda \cdot P_R$$

3. Implementação

Devido à natureza dinâmica e não determinística do problema, e à grande complexidade que envolveria tratá-lo como um único problema de procura global sobre todos os estados, optou-se por tomar uma abordagem híbrida a nível de gestão de recursos e a nível de determinação das rotas. Para a atribuição de pedidos e recarga/abastecimento dos veículos, desenvolveu-se um algoritmo iterativo que atualiza o estado do sistema a cada instante e recorre ao uso de algoritmos de procura para calcular as rotas entre duas localizações. Este algoritmo trata de rejeitar os pedidos cujo tempo máximo de espera tenha passado, selecionar, para cada pedido pendente, o veículo que esteja nas melhores condições para o atender, caso exista, e, dos veículos que restem disponíveis, recarregar ou abastecer os que possuam uma autonomia menor ou igual a 20% da sua autonomia máxima.

Para a atribuição de um veículo a cada pedido é utilizada uma heurística de seleção, sendo filtrados inicialmente os veículos viáveis, tendo em conta a preferência ambiental do pedido, a capacidade de passageiros e a autonomia atual do veículo em comparação com limites inferiores de distância obtidos através do cálculo da distância euclidiana entre duas localizações. Estes veículos viáveis são posteriormente ordenados com base numa heurística de custo que leva em consideração o impacto ambiental dos veículos, uma aproximação da distância a percorrer e o tempo estimado para a percorrer. Tendo o conjunto de veículos ordenados, percorre-se este conjunto até se encontrar um veículo com autonomia suficiente para percorrer as rotas calculadas através dos algoritmos de procura. As rotas determinadas nesta fase incluem a da localização atual do veículo até à origem do pedido, da origem até ao destino e do destino até ao ponto de abastecimento/recarga disponível mais próximo, garantindo que eles possuam uma autonomia de reserva para deslocarem-se para esses nós após a viagem, caso necessário, de forma a não deixar o sistema num estado comprometido.

Para a procura de rotas no grafo que representa o mapa da cidade foram implementados os algoritmos de procura a seguir descritos, de forma a compará-los com base na otimalidade das soluções obtidas e no custo a nível de desempenho computacional.

3.1. Algoritmos de procura não informada

3.1.1. Bfs (Procura em largura)

Este algoritmo percorre o mapa camada a camada, identificando o caminho entre a origem e o destino com o menor número de saltos (nós intermédios). Foi utilizada a estrutura *deque* para gerir a fila dos nós a visitar. Assim, podemos usar operações como *popleft* em tempo constante, o que é significativamente mais eficiente do que as listas nativas para manter o método FIFO.

Complexidade Temporal:¹ $O(V+E)$

Complexidade Espacial: $O(V)$

3.1.2. Dfs (Procura em profundidade)

Este algoritmo explora o grafo seguindo cada ramo até à sua profundidade máxima antes de retroceder (*backtracking*). A implementação utiliza uma pilha (*stack*) explícita para gerir os nós a explorar, onde cada elemento da pilha contém tanto o nó atual como o caminho percorrido desde a origem. Inicia-se com o nó de partida e explora recursivamente os seus vizinhos, armazenando os nós visitados num conjunto (*set*) para evitar ciclos. Quando o nó destino é encontrado, o caminho completo é reconstruído e o custo total é calculado.

Complexidade Temporal: $O(V+E)$

Complexidade Espacial: $O(V)$

A complexidade temporal aumenta proporcionalmente à soma do número de vértices e arestas, e a complexidade espacial é proporcional ao número de vértices.

3.1.3. Iterativo

Este algoritmo percorre o mapa em profundidade, tal como o algoritmo DFS, mas distingue-se por operar com uma profundidade máxima definida que é incrementada iterativamente. Desta forma, o algoritmo evita percorrer caminhos infinitos e garante que a solução encontrada corresponde ao caminho entre a origem e o destino com o menor número de saltos.

Para a gestão de ciclos, o algoritmo recorre a um mecanismo de *backtracking* utilizando um conjunto (*set*). Quando um nó é visitado, é adicionado a este conjunto para impedir que seja reprocessado nesse ramo e assim que a exploração dos seus vizinhos termina, o nó é removido do conjunto. Esta abordagem, permite que este nó possa ser considerado noutros caminhos alternativos.

Complexidade Temporal:² $O(b^d)$

Complexidade Espacial: $O(bd)$

3.1.4. Custo Uniforme

Este algoritmo expande sempre o nó com menor custo acumulado desde a origem, garantindo a otimalidade do caminho encontrado. A implementação utiliza uma fila de prioridade (*min-heap*) para gerir eficientemente a fronteira de exploração, processando em primeiro lugar os caminhos com menor custo.

Complexidade Temporal: $O(E \log(V))$

Complexidade Espacial: $O(V)$

Desta forma, a complexidade temporal tende a aumentar proporcionalmente ao número de arestas no grafo, enquanto a complexidade espacial é proporcional ao número de vértices.

¹ V : Representa o número total de nós existentes no grafo.

E : Representa o número total de Arestas.

² b : Refere-se ao número médio de sucessores de cada nó.

d : Indica a profundidade da nó solução mais próxima do objetivo.

3.2. Algoritmos de procura informada

As heurísticas são funções utilizadas pelos algoritmos de procura informada (A* e *Greedy*) para estimar o caminho mais curto entre o nó atual e o nó destino.

(H_d) - heurística da distância de manhattan

Esta heurística calcula o custo entre dois pontos baseando-se na soma das diferenças absolutas das suas coordenadas cartesianas. Cálculo:

$$distance_heuristic = |x_1 - x_2| + |y_1 - y_2|$$

(H_t) - heurística da distância do tempo estimado

Utilizando a distância de Manhattan, divide-a pela velocidade do veículo para prever quanto tempo levará para chegar ao destino. Cálculo:

$$time_heuristic = \frac{\text{Distância de Manhattan}}{\text{Velocidade do Veículo}}$$

3.2.1. A Star

Este algoritmo de procura informada combina o custo do caminho percorrido com uma estimativa heurística do custo restante até ao destino.

A implementação utiliza duas estruturas principais: uma lista para nós a explorar (*open_list*) e outra para nós já processados (*closed_list*). Em cada iteração, é selecionado o nó com menor valor da soma do custo do caminho e heurística para expansão. Os seus vizinhos são então examinados. Se um vizinho não está em nenhuma das listas, ainda não foi explorado e é adicionado à lista aberta. Caso contrário, se já foi visitado mas é encontrado um caminho melhor até ele, o seu custo é atualizado e, caso esteja na lista fechada, é reaberto e movido de volta para a lista aberta. Este ciclo repete-se até que o nó destino seja alcançado ou até que não haja mais nós a explorar, garantindo que o primeiro caminho completo encontrado é o ótimo.

Complexidade Temporal: $O(b^d)$

Complexidade Espacial: $O(V)$

Esta abordagem combina a garantia de otimização do custo uniforme com a eficiência da procura gulosa, explorando prioritariamente os caminhos mais promissores. Desta forma, no pior caso, o tempo de execução tende a aumentar proporcionalmente ao número de arestas no grafo, e a memória necessária é proporcional ao número de vértices.

3.2.2 Greedy

O algoritmo greedy seleciona, em cada passo, o nó que aparenta estar mais próximo do destino baseando-se numa função heurística. Ao contrário do algoritmo A*, esta abordagem ignora o custo do caminho já percorrido, e não contabiliza o peso das arestas durante o processo de seleção do próximo nó, focando-se apenas na estimativa do custo restante até ao objetivo.

Complexidade Temporal:³ $O(b^m)$

Complexidade Espacial: $O(V)$

³ m : Representa a profundidade máxima do caminho mais longo possível.

Na implementação realizada, o algoritmo percorre o conjunto de nós não explorados para escolher o nó com a menor heurística. Esta estratégia resulta numa solução rápida, mas esta pode ser longa e dispendiosa uma vez que toma decisões baseadas numa estimativa sem considerar o custo real.

4. Testes

4.1. Comparação dos algoritmos de procura

Para comparação dos algoritmos de procura foi utilizado um conjunto fixo de 15 pedidos representado em `data/pedidos.json`, de forma a garantir que os algoritmos fossem testados nas mesmas condições. As métricas de comparação recolhidas nas diferentes execuções do programa incluem o tempo de execução acumulado do algoritmo, o tempo simulado de viagem total em minutos, a distância acumulada, o número acumulado de nós nos caminhos encontrados e de nós visitados. Também se realizaram diferentes execuções do Greedy e do A* para cada heurística implementada. Os resultados dos testes realizados encontram-se representados na tabela 2.

Algoritmo	Tempo	Custo	Distância	Nós	Nós Visitados
BFS	0.8304	200.66	111.59	75	237
DFS	1.7901	281.42	161.45	98	798
Iterativo	15.0825	205.75	113.51	75	1142
Custo Uniforme	2.9641	196.78	116.05	81	301
Greedy (H_d)	1.4105	207.36	112.04	79	348
Greedy (H_t)	1.4205	207.36	112.04	79	348
A* (H_d)	2.4549	207.36	112.04	79	348
A* (H_t)	3.2237	188.77	104.11	78	362

Tabela 2: Resultados obtidos nos testes de comparação de algoritmos de procura

4.2. Teste

De forma a avaliar o desempenho geral do sistema para diferentes cenários de teste, desenvolveu-se um método que gera um certo número de pedidos aleatoriamente ao longo de uma dada janela de tempo simulado, presente em `pedido_gerador.py`. De notar que, diferentemente dos pedidos utilizados para testar os algoritmos de procura, os valores dos campos dos pedidos gerados nesta abordagem (origem, destino, número de passageiros, tempo máximo de espera, horário

pretendido, prioridade e preferência ambiental) são variáveis, encontrando-se alguns entre os seguintes limites:

- número de passageiros: 1 a 4
- tempo máximo de espera: 5 a 30 minutos
- horário pretendido: 0 a 60 minutos da hora simulada atual

A diferença entre a hora da chegada dos pedidos e o horário pretendido também varia consoante estes valores e o número de pedidos. Assim, o custo obtido para diferentes execuções do mesmo número de pedidos depende de vários fatores aleatórios, pelo que, para garantir uma menor discrepância entre os resultados, os valores do custo de cada linha da tabela abaixo representada correspondem a uma média dos valores obtidos para 3 execuções diferentes do programa.

O algoritmo utilizado foi o A* com a heurística H_f e a janela de tempo simulado para chegada dos pedidos foi de 4 horas. Para o cálculo do custo total, os pesos de cada fator utilizados foram os seguintes:

- $\alpha = 1$
- $\beta = 0.1$
- $\gamma = 0.2$
- $\delta = 0.15$
- $\lambda = 2$

P	O (€)	T (minutos)	E (kg/km)	D (km)	P_R	C
15	12.63	3.95	3.78	93.01	0	28
30	34.31	12.63	18.69	193.48	0	68
50	50.50	121.91	31.44	263.13	6	121
100	81.76	869.67	59.74	381.09	40	318

Tabela 3: Resultados obtidos nos testes de comparação de algoritmos de procura

Os valores obtidos para os testes são esperados. Até 50 pedidos, o sistema consegue lidar bem com a chegada dos pedidos, verificando-se um número de pedidos rejeitados relativamente pequeno. Já com 100 pedidos, o sistema fica sobrecarregado, observando-se um disparo no número de pedidos rejeitados. Este aumento também se reflete no tempo acumulado de espera total, dado que, embora também dependa do número de pedidos, torna-se significativamente maior para um número maior de pedidos rejeitados já que nestes casos vai corresponder ao tempo máximo de espera definido.

5. Interface gráfica

Para permitir a visualização do comportamento dos algoritmos e a evolução do estado do sistema em tempo real, foi desenvolvido um módulo gráfico utilizando a biblioteca Pygame.

A representação visual dos elementos do sistema segue a seguinte codificação:

- **Infraestrutura (Grafo):** Os nós são diferenciados por cores consoante a sua função: pontos de recolha (Azul), postos de combustão (Verde) e estações de carregamento (Laranja).
- **Veículos:** A frota é representada por ícones móveis que alteram a sua cor ou aspeto visual de acordo com o estado atual: disponíveis (Verde/Amarelo), ocupados com passageiro (Vermelho) ou em abastecimento (Azul).
- **Ambiente:** As arestas representam as vias de comunicação, incluindo uma visualização da densidade de trânsito variável e diferenciação por tipo de zona (ex: Old Town ou Residencial).

Adicionalmente, a interface apresenta um painel de controlo (HUD) com métricas em tempo real, como o progresso dos pedidos e a hora simulada, facilitando a validação visual das soluções encontradas pelos algoritmos de procura.

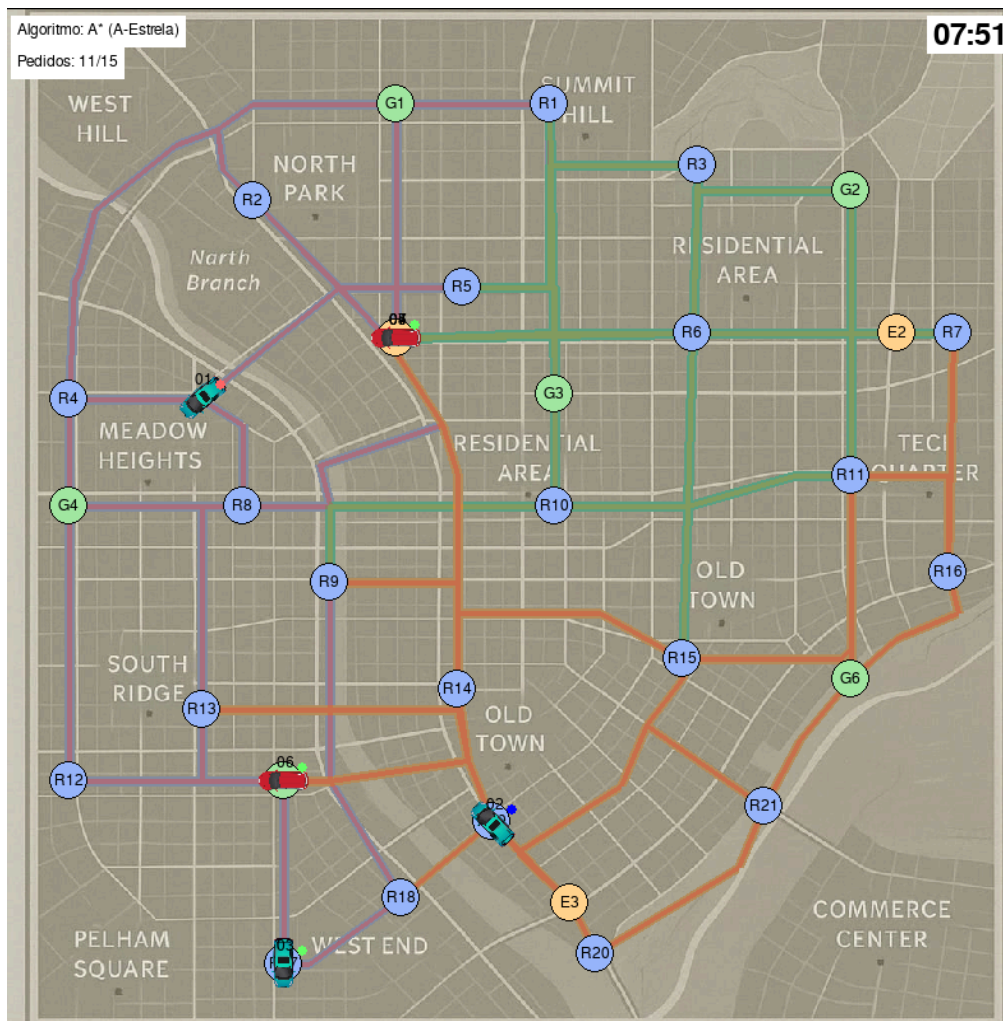


Figura 2: Visualização gráfica do grafo em tempo real

6. Conclusão

O desenvolvimento deste sistema permitiu consolidar a aplicação prática de algoritmos de procura em cenários de contingência e otimização multivariável. A análise comparativa demonstrou inequivocamente que, em espaços de estados complexos, a utilização de estratégias de procura informada (nomeadamente o A*) é determinante para a viabilidade da solução, uma vez que o uso de heurísticas adequadas permite direcionar a exploração e reduzir drasticamente o custo computacional face às abordagens cegas.

Mais do que apenas encontrar caminhos, o projeto evidenciou o compromisso necessário entre a otimalidade matemática e a eficiência temporal. Enquanto algoritmos como o Greedy oferecem rapidez sacrificando a qualidade da solução, o A* (com heurística temporal) provou ser o ponto de equilíbrio ideal para um ambiente dinâmico.

Por fim, os testes demonstraram uma limitação importante: os algoritmos conseguem gerir melhor a frota, mas não resolvem a falta de recursos físicos. Quando a procura é muito elevada, o sistema atinge o seu limite simplesmente porque não há carros ou estações suficientes. Isto prova que não basta ter um bom *software*; é fundamental ter também uma frota e infraestrutura ajustadas à quantidade de pedidos, pois a Inteligência Artificial não consegue compensar a falta de veículos.